University of Agriculture Faisalabad

Department of Computer Science
Faculty of Sciences

TICKET BOOKING SYSTEM

TECHNICAL REPORT

SUBMITTED BY

MUHAMMAD ARHAM ZAHID (2024-AG-4173)

ADVISED BY

Sir . QAMAR NAWAZ

A TECHNICAL REPORT SUBMITTED IN PARTIAL FULFILLMENT OF
REQUIREMENT FOR THE DEGREE OF

BACHELOR OF SCIENCE
IN
COMPUTER SCIENCE

2025-2026

DECLARATION

I hereby declare that the contents of the report "Ticket Booking System" is a project of my own and no part has been copied from any published source (except the references). I further declare that this work has not been submitted for award of any other diploma/degree. The university may take action if the information provided is found false at any stage. In case of any default the scholar will be proceeded against as per UAF policy.

_____

Student Name

Ag no.

CERTIFICATE

To,

The Controller of Examinations,

University of Agriculture,

Faisalabad.

The supervisory committee certifies that [Student Name] bearing AG # [XXXX] has successfully completed his/her project "Ticket Booking System" in partial fulfillment of requirement for the degree of B.Sc. Computer Science under our guidance and supervision.

_____

Sir. Qamar Nawaz

Department of Computer Science

_____

Dr. Muhammad Ahsan Latif

Chairman,

Department of Computer Science

ACKNOWLEDGEMENT

I thank all who in one way or another contributed to the completion of this report. I am grateful to the Department of Computer Science, University of Agriculture Faisalabad, for providing the resources and environment to work on this project. My heartfelt thanks go to my supervisor for the continuous guidance, encouragement, and valuable suggestions throughout the development of this system. I am also thankful to my fellow students for their constructive feedback and criticism which helped improve the quality of the work. Finally, I express my gratitude to my family for their constant support and prayers. May Allah bless all of them.

ABSTRACT

The Ticket Booking System is a Windows desktop application developed using C# and .NET 10 with SQL Server as the back-end database. The system is designed to automate and simplify the process of booking cinema tickets for customers while providing an administrative interface for cinema staff to manage movies, bookings, and seat availability.

The system distinguishes between two types of users: general users and administrators. A general user can register an account, log in securely, browse available movies, select seats visually on a seat map, choose a ticket category (VIP, Gold, or Silver), review the booking summary, and confirm payment through a payment form. The system automatically calculates the total cost based on the selected category and number of tickets and prevents double-booking by marking already reserved seats in red.

The administrator can log in through a dedicated admin account and access management features including adding, editing, and deleting movies with their show times, viewing and deleting customer bookings, and monitoring seat occupancy for any movie and time slot. The system uses parameterized SQL queries throughout, providing protection against SQL injection attacks. Case-sensitive authentication is enforced using SQL collation to ensure secure login. All booking data is persisted in a relational SQL Server database consisting of three tables: Users, Movies, and Bookings. The system was developed using the Agile methodology with iterative development cycles, allowing progressive refinement of features based on testing and feedback.

CHAPTER 1 – INTRODUCTION

1.1 Background

The entertainment industry, particularly the cinema sector, has grown significantly over the past decade. Traditional manual ticketing methods are slow, error-prone, and difficult to scale. Long queues, double-bookings, and the inability to view seat availability in real time are common problems faced by both cinema operators and moviegoers. The introduction of computerized ticketing systems has transformed how cinemas operate, enabling fast bookings, automated seat management, and centralized data storage.

In Pakistan, most small and medium-sized cinema halls still rely on manual counter-based ticketing or very basic point-of-sale systems. These systems lack features such as visual seat selection, category-based pricing, or online payment methods. The Ticket Booking System developed in this project addresses this gap by providing a fully functional desktop-based cinema booking application suitable for local cinema deployment. The system is built on widely available technology (C#, .NET, and SQL Server) ensuring low cost and easy maintainability.

1.2 Description

The Ticket Booking System is a desktop application developed in C# using Windows Forms (.NET 10). It provides a complete workflow from user registration and login through movie selection, visual seat selection, payment confirmation, and booking history viewing. The system has a separate administrative module accessible only to authorized admin users.

The user-facing side of the system allows customers to create an account and log in with case-sensitive credentials. After login, they are taken to the main menu from which they can initiate a new booking. The booking form allows selection of a movie from the database, automatic population of the show time, selection of ticket quantity (using a numeric spinner), and selection of ticket category (VIP at 800 PKR, Gold at 600 PKR, Silver at 400 PKR). The total price is calculated automatically. Users then proceed to an interactive seat map where available seats are shown in light gray, booked seats in red, and selected seats turn green. Once seats are confirmed, the user proceeds to the payment form which shows a summary of the booking and allows selection of a payment method before final confirmation.

The admin side provides a dashboard with menu items for managing movies (add, edit, delete with show times), managing bookings (view all, edit customer name, delete), and viewing the seat status for any movie and time slot. The admin can also clear all seats for a given show (cancel all bookings for that session).

1.3 Problem Statement

Cinema halls in many localities rely on manual or semi-automated ticketing systems that result in frequent double-bookings, inability to view seat availability before purchase, long processing times at counters, and poor record keeping. There is a need for a reliable, low-cost desktop application that can be deployed at a cinema counter to manage ticket sales, seat occupancy, and movie schedules in a centralized and automated manner.

1.4 Scope

The Ticket Booking System covers the following scope:

- User registration and secure login.

- Movie and show time management by administrators.
- Visual seat selection with real-time availability from the database.
- Category-based ticket pricing (VIP, Gold, Silver).
- Payment method selection and booking confirmation.
- Booking history viewing for individual users.
- Admin management of movies, bookings, and seat occupancy.

The system does not cover online/web-based booking, SMS/email notifications, or integration with payment gateways.

1.5 Objectives

- Develop a fully functional desktop cinema ticket booking system.
- Implement secure user authentication with case-sensitive login.
- Provide an interactive visual seat map linked to the live database.
- Support category-based dynamic pricing for tickets.
- Allow administrators to manage movies and bookings efficiently.
- Store all data in a relational SQL Server database (Users, Movies, Bookings).

1.6 Feasibility

1.6.1 Technical Feasibility

The system uses C# and Windows Forms, a mature and well-supported technology. SQL Server provides reliable data storage with robust querying capabilities. The development tools (Visual Studio) are freely available. The technology stack is well within the capability of a university-level development team.

1.6.2 Operational Feasibility

The system is designed for use at a cinema counter by trained staff. The interface is straightforward and does not require significant technical knowledge to operate. Admin functions are restricted by a separate login to prevent unauthorized access.

1.6.3 Economic Feasibility

The system is developed using free development tools and a SQL Server Express instance (free tier). Deployment requires only a standard PC already present at a cinema counter, making the cost of adoption minimal.

1.7 Requirements

1.7.1 Functional Requirements

- Users shall be able to register with a unique username and password.
- Users shall be able to log in with case-sensitive credentials.
- Users shall be able to select a movie and show time from the database.
- Users shall be able to select seats visually with real-time availability.
- The system shall calculate total price based on ticket count and category.
- Users shall be able to confirm payment and save bookings to the database.

- Admins shall be able to add, edit, and delete movies.
- Admins shall be able to view, edit, and delete any booking.
- Admins shall be able to view and clear seat status for any show.

1.7.2 Non-Functional Requirements

- The system shall enforce case-sensitive login using SQL collation.
- All SQL queries shall use parameterized commands to prevent SQL injection.
- The seat map shall reflect the live database state on form load.
- The system shall display appropriate error messages for invalid inputs.

1.7.3 Hardware Requirements

- Processor: Intel Core i3 or equivalent (minimum).
- RAM: 4 GB minimum.
- Storage: 500 MB free disk space.
- Display: 1024 x 768 resolution minimum.

1.7.4 Software Requirements

- Operating System: Windows 10 or higher.
- .NET Runtime: .NET 10 (Windows).
- Database: Microsoft SQL Server (Express or higher).
- IDE: Visual Studio 2022 (for development).

1.8 Stakeholders

- Cinema Customers: register, book tickets, view booking history.
- Cinema Counter Staff: assist customers with bookings.
- Cinema Administrator: manage movies, bookings, and seat data.
- System Developer: develop and maintain the application.

CHAPTER 2 – MATERIALS AND METHODS

2.1 Process Model

The Agile Software Development Methodology was adopted for this project. Agile allows for iterative development with short cycles called sprints. Each sprint delivered a working increment: Sprint 1 covered user authentication (Form1(login), Form2(registration)), Sprint 2 covered the booking workflow (Form3(main menu), bookingform, seatselection), Sprint 3 covered payment and view bookings, and Sprint 4 covered the admin module. Testing and refinement were performed at the end of each sprint.Also I choose this model because I need to fix designs ,bugs or shoud add new features later in the development .

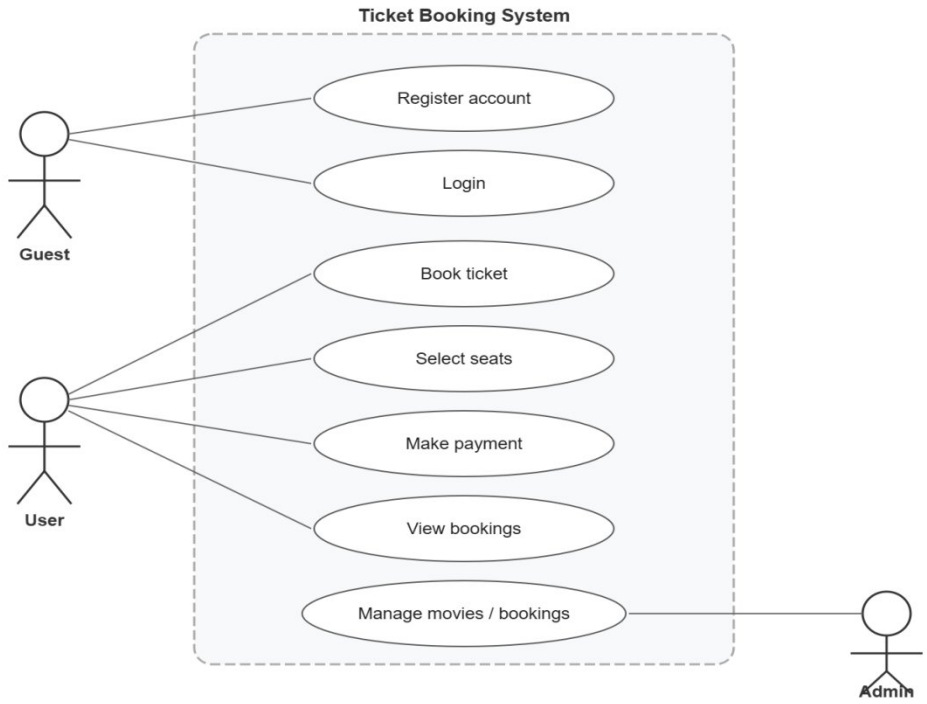
2.2 Tools and Technologies

- Programming Language: C# (.NET 10, Windows Forms)
- Database: Microsoft SQL Server with Microsoft.Data.SqlClient
- IDE: Visual Studio 2022
- Version Control: Git (GitHub)
- Documentation: WPS
- Diagram Tool: Draw.io / Manual SVG

2.3 Design

2.3.1 Use Case Diagram

The use case diagram below shows the interactions between the three main actors (Guest, User, Admin) and the system. A Guest can register or log in. A logged-in User can book tickets, select seats, make payment, and view their booking history. The Admin can manage movies, bookings, and seat status.



Use Case Title	1. User Booking System
Use Case Id	1
Requirement Id	3
Description:	This use case describes the overall process where a registered user books movie tickets. It includes login, selecting movie and showtime, selecting seats, making payment and viewing the booking confirmation.

Pre Conditions:

1. User must be registered and logged in.
2. Movies and showtimes must be available in the system.
3. Seats must be available for the selected show.

Task Sequence	Exceptions
1. User logs into the system with valid credentials.	Invalid username or password.
2. User selects a movie and showtime.	Selected movie or showtime not available.
3. System displays available seats.	
4. User selects the required number of seats.	Selected seats are already booked.
5. System calculates the total amount.	
6. User proceeds to payment and confirms the payment.	Payment cancelled or failed.
7. System saves the booking details in the database.	Database error while saving booking.
8. System displays booking confirmation to the user.	

Use Case Title	2. Admin Management System
Use Case Id	2
Requirement Id	6
Description:	This use case describes the overall process where the administrator manages movies, bookings and seat status in the system.

Pre Conditions:

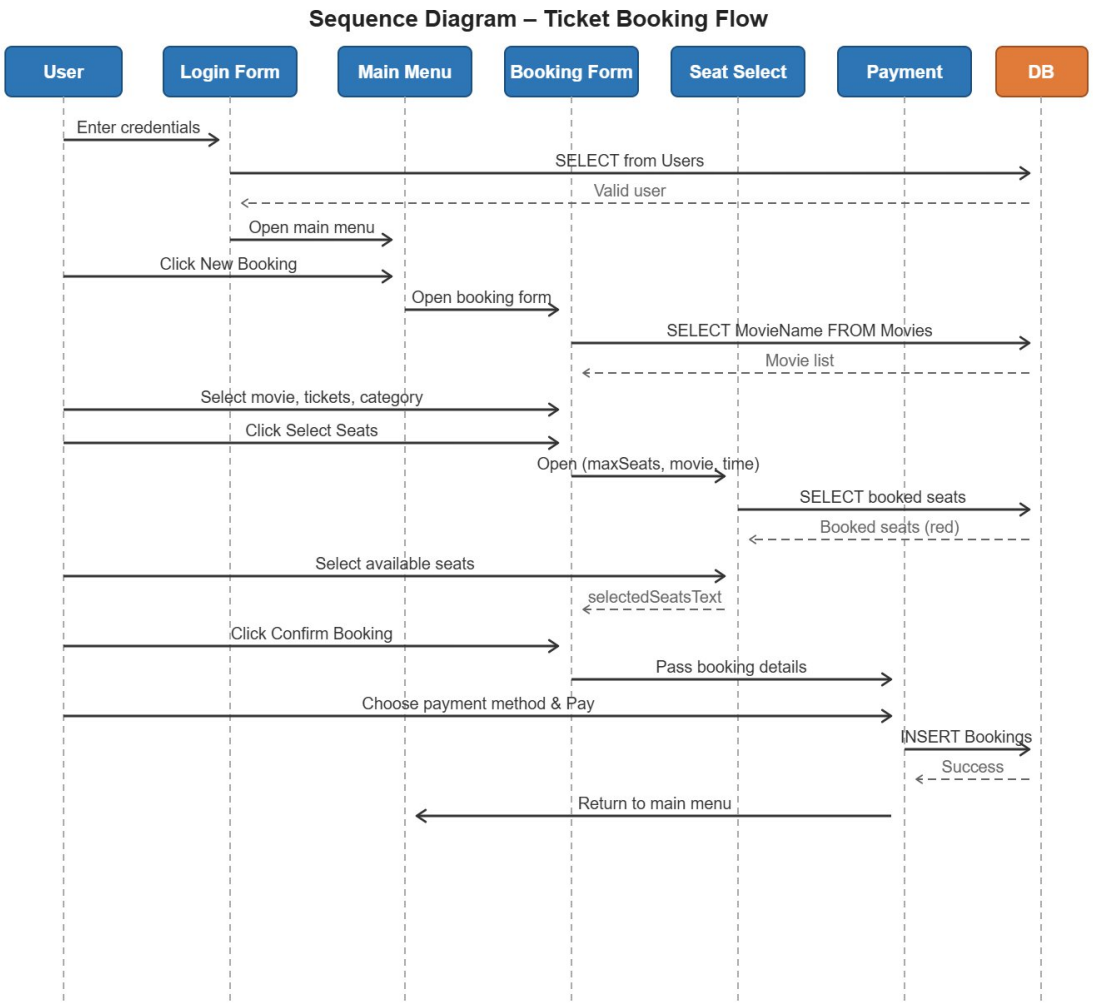
1. Administrator must be logged in.
2. Database must be available in online mode.

Task Sequence	Exceptions
1. Admin opens the management panel.	
2. Admin selects the required module (Movies / Bookings / Seats).	
3. Admin adds, updates or deletes the records.	Invalid data entered.
4. System validates the data.	Validation error.
5. System updates the database.	Database update failure.
6. Updated records are displayed to the administrator.	
7. Admin logs out from the system.	

Use Case Title	3. User Authentication
Use Case Id	3
Requirement Id	1
Description:	This use case describes how users register a new account or login to an existing account.
Pre Conditions: 1. Database must be available in online mode.	
Task Sequence	**Exceptions**
1. User selects to register or login.	
2. User enters the required information.	Incomplete or invalid information.
3. System validates the information.	Username already exists / Invalid credentials.
4. For registration: System creates a new account.	
5. For login: System grants access to the main menu.	Database connection failure.

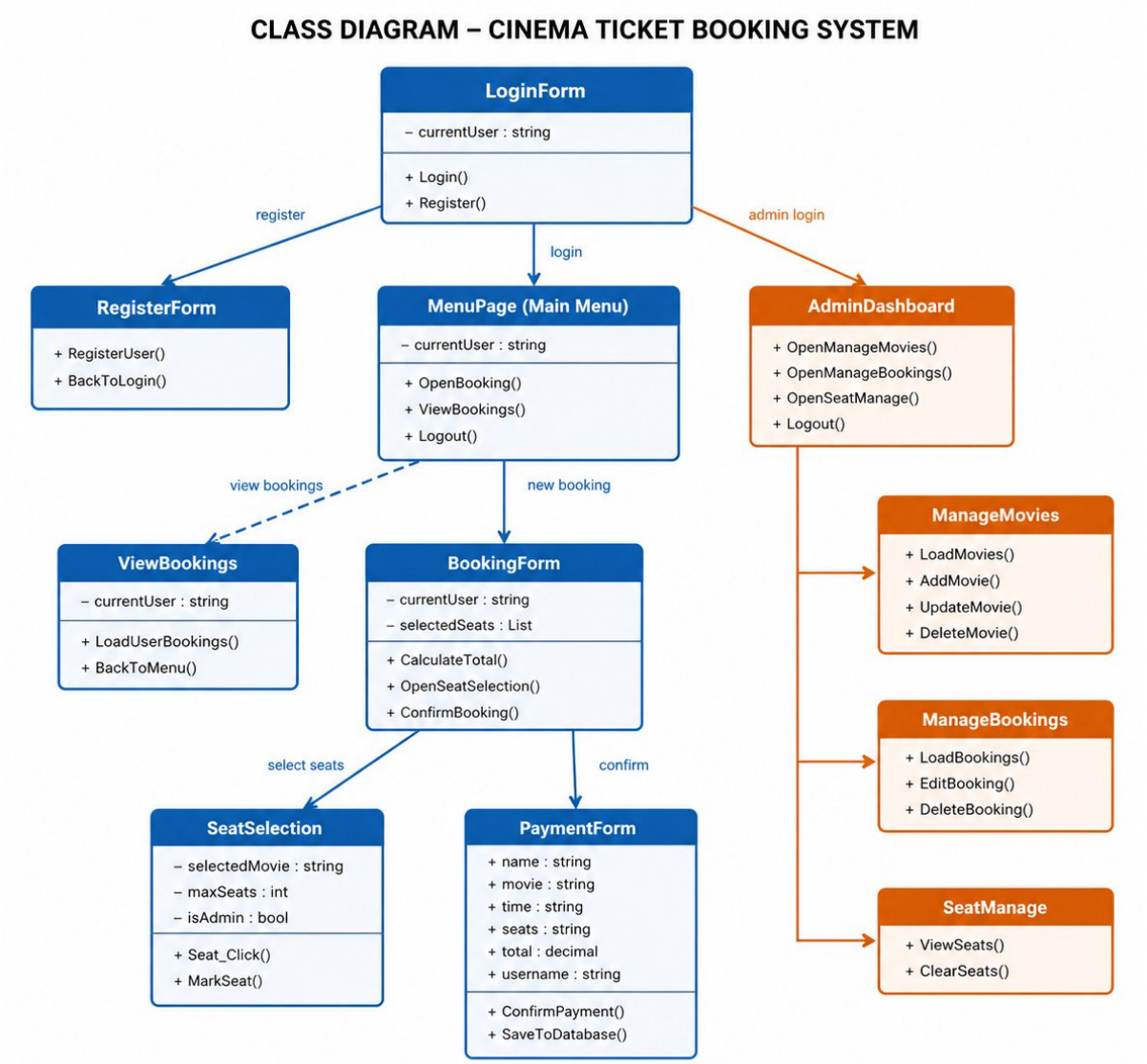
2.3.2 Sequence Diagram

The sequence diagram illustrates the booking workflow: the user logs in, navigates to the booking form, selects a movie (fetched from the database), opens the seat selection form (which loads booked seats from the database), selects seats, confirms, and proceeds to payment. Payment writes the booking record to the Bookings table.



2.3.3 Class Diagram

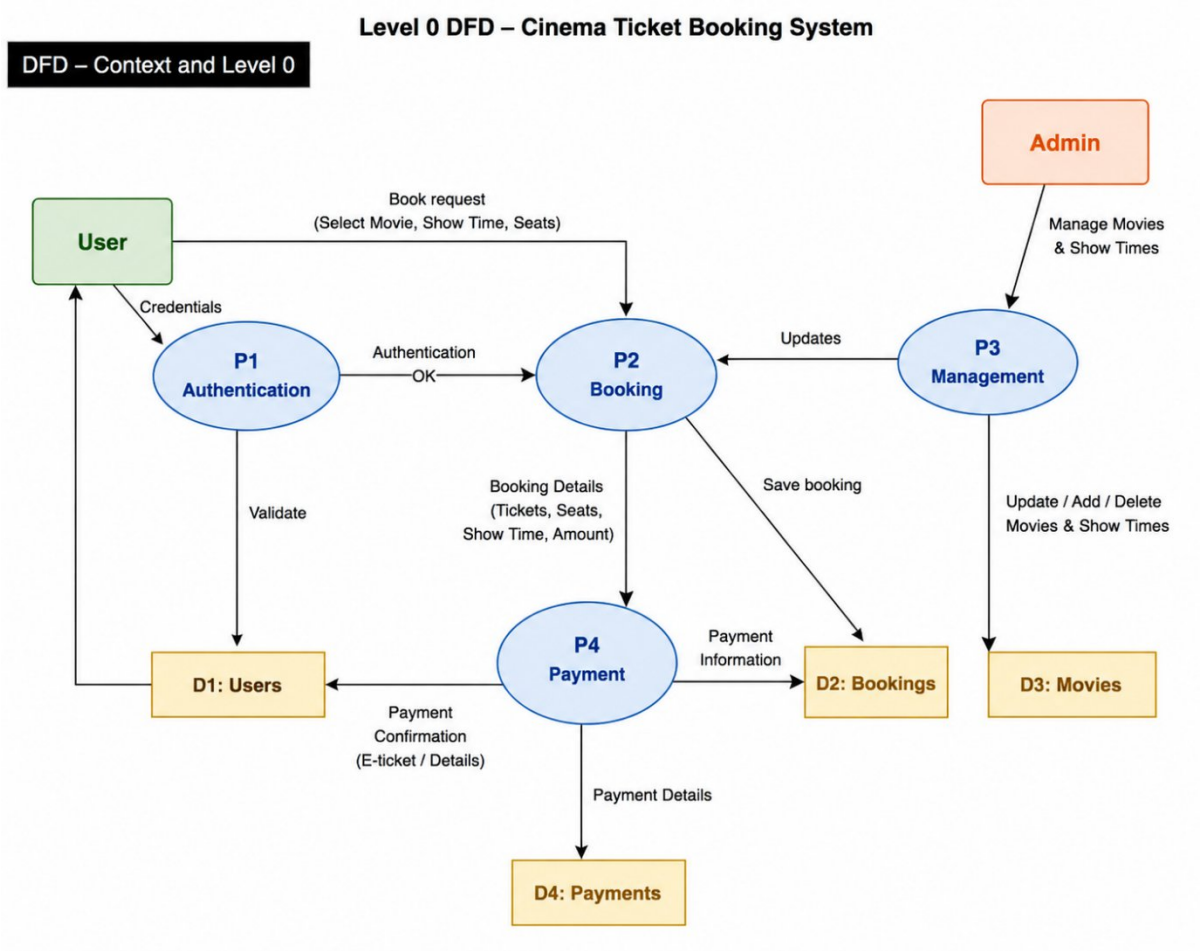
The class diagram shows all Windows Forms classes in the application and their relationships. Key relationships include Form1(login) navigating to Form3(main menu) (on valid login) or Admin (for admin user); bookingform opening seatselection as a dialog and then payment; Admin navigating to ManageMovies, ManageBookings, and Seatmanage.



[2.3.4 Data Flow Diagram

The Context Diagram (Level 0 DFD) shows the system as a single process with two external entities: User and Admin. Users send login/booking requests and receive confirmations. The Admin sends management commands and receives reports.

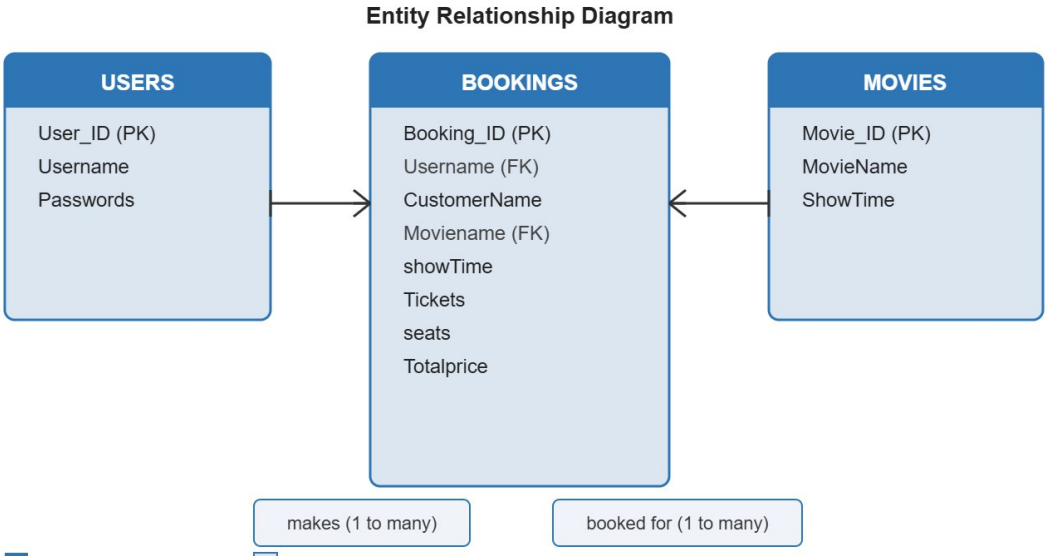
The Level 1 DFD decomposes the system into three processes: P1 Authentication (validates credentials against the Users table), P2 Booking (records bookings in the Bookings table and reads movie data from the Movies table), and P3 Management (admin CRUD on Movies and Bookings tables).



2.3.5 ER Diagram

The Entity-Relationship Diagram shows three entities: USERS (User_ID PK, Username, Passwords), MOVIES (Movie_ID PK, MovieName, ShowTime), and BOOKINGS (Booking_ID PK, Username FK, CustomerName, Moviename FK, showTime, Tickets, seats, Totalprice).

A User makes zero or more Bookings. A Movie is associated with zero or more



2.3.6 Database Model

The physical database consists of the following three tables in the ticketdb SQL Server database:

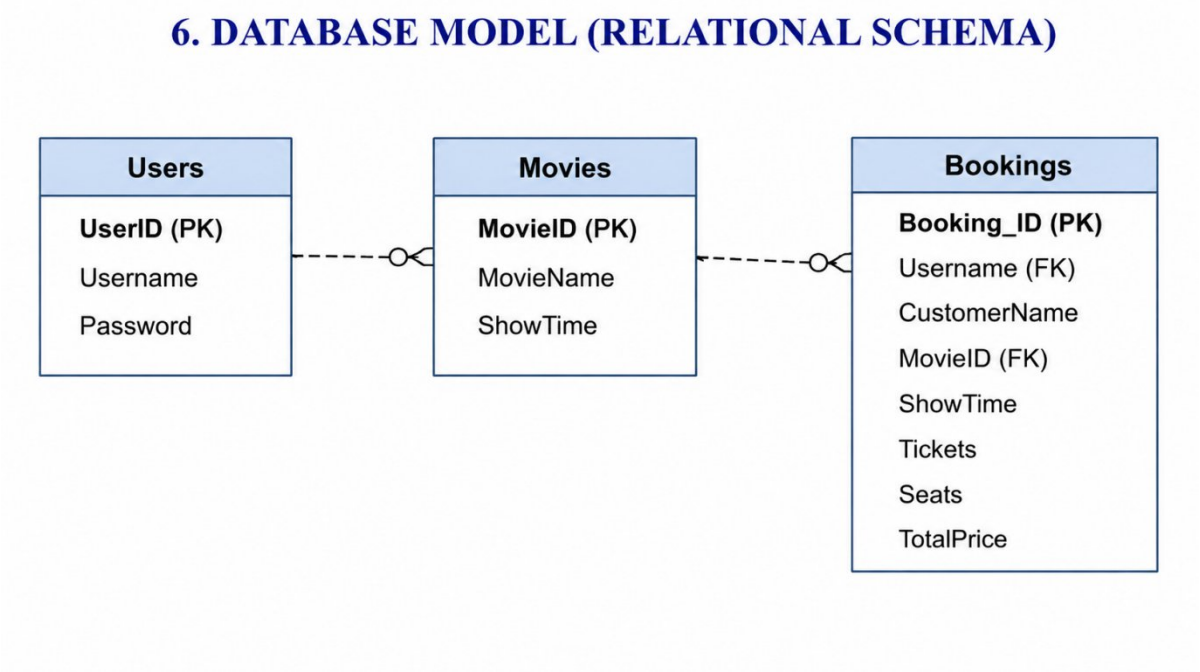


Table 2.1: Users

Column	Data Type	Constraint	Description
User_ID	INT	PK, IDENTITY	Unique user identifier
Username	VARCHAR(100)	NOT NULL	Login username

Passwords	VARCHAR(100)	NOT NULL	Login password

Table 2.2: Movies

Column	Data Type	Constraint	Description
Movie_ID	INT	PK, IDENTITY	Unique movie identifier
MovieName	NVARCHAR(200)	NOT NULL	Name of the movie
ShowTime	NVARCHAR(50)	NOT NULL	Show time string

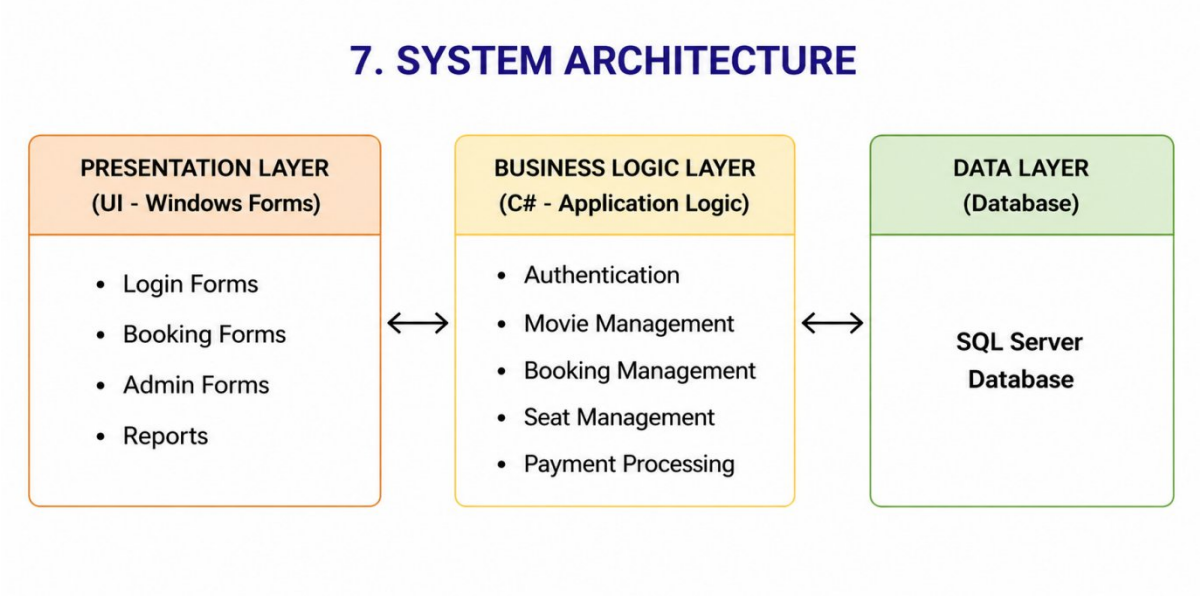
Table 2.3: Bookings

Column	Data Type	Constraint	Description
Booking_ID	INT	PK, IDENTITY	Unique booking identifier
Username	VARCHAR(100)	FK -> Users	Booked by user
CustomerName	VARCHAR(200)	NOT NULL	Customer display name
Moviename	VARCHAR(200)	FK -> Movies	Movie booked
showTime	VARCHAR(50)	NOT NULL	Screening time
Tickets	INT	NOT NULL	Number of tickets
seats	VARCHAR(500)	NOT NULL	Comma-separated seat IDs
Totalprice	INT	NOT NULL	Total price in PKR

2.3.7 System Architecture

The system follows a 2-tier client-server architecture. The presentation layer consists of Windows Forms screens running on the client machine. The data layer is a SQL Server database (ticketdb) accessed directly via Microsoft.Data.SqlClient. There is no separate business logic tier; validation and business rules are enforced in the form event handlers. This architecture is appropriate for a single-station deployment at a cinema counter.

- Presentation Layer: Windows Forms (Form1, Form2, Form3, bookingform, seatselection, payment, viewBooking, Admin, ManageMovies, ManageBookings, Seatmanage)
- Data Access Layer: SqlConnection, SqlCommand, SqlDataAdapter (Microsoft.Data.SqlClient)
- Database: SQL Server (ticketdb) with tables Users, Movies, Bookings



CHAPTER 3 – RESULTS AND DISCUSSION

3.1 Testing

The system was tested using Black Box Testing techniques. Each module was tested independently with valid and invalid inputs to verify correct behavior. Test cases were designed to cover normal flows, boundary conditions, and error conditions.

3.2 Test Cases

Table 3.1: User Login Test Case

Test ID	Input	Expected Output	Actual Output	Status
TC-01	Valid username & password	Login successful, navigate to Form3	Login successful	Pass
TC-02	Wrong password	Incorrect Password message	Incorrect Password!	Pass
TC-03	Non-existent username	Username not found message	Username not found!	Pass
TC-04	admin / 123123	Navigate to Admin form	Admin panel opened	Pass
TC-05	Empty fields	Please enter credentials message	Error displayed	Pass

Table 3.2: Booking Test Case

Test ID	Input	Expected Output	Actual Output	Status

TC-06	Complete booking details + seats	Booking confirmed, record in DB	Booking Confirmed!	Pass
TC-07	Book without selecting seats	Error: select seats first	Please select seat(s) first!	Pass
TC-08	Book without selecting movie	Error: select a movie	Please select a movie!	Pass
TC-09	Select already booked seat	Seat not selectable (red)	Seat marked red, click ignored	Pass
TC-10	Select more seats than tickets	Error: cannot select more	Warning message shown	Pass

Table 3.3: Admin Module Test Case

Test ID	Input	Expected Output	Actual Output	Status
TC-11	Add new movie with time	Movie added to grid	Movie Added!	Pass
TC-12	Delete booking (Yes confirm)	Booking removed from grid	Booking deleted!	Pass
TC-13	Clear seats for a show	All bookings for show deleted	Seats cleared!	Pass

3.3 Conclusion

The Ticket Booking System has been successfully developed and tested. All planned features were implemented, including secure user authentication, dynamic seat selection linked to the database, category-based pricing, payment confirmation, and a complete admin management module. Testing confirmed that the system correctly handles both valid and invalid inputs and provides appropriate feedback to users.

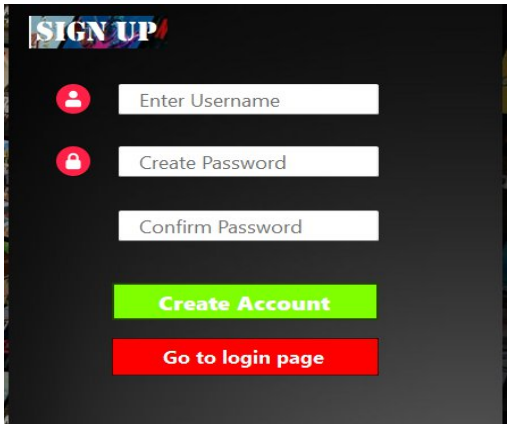
The system demonstrates the practical application of C# Windows Forms, SQL Server, and Agile development methodology. Future enhancements could include a web-based version, email confirmation of bookings, online payment gateway integration, and multi-screen cinema support.

CHAPTER 4 – USER MANUAL

4.1 Getting Started

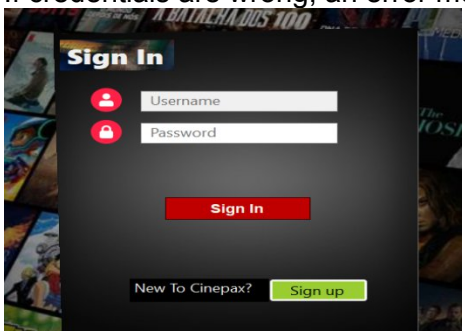
Ensure that SQL Server is running and the ticketdb database has been created with the required tables (Users, Movies, Bookings). Update the connection string in the source code to match your SQL Server instance name. Build and run the solution in Visual Studio 2022.

4.2 User Registration

- Launch the application. The Login form (Form1) appears.
- Click the Register button to open the Registration form (Form2).
- Enter a unique username, password, and confirm password.
- Click Register. A success message will appear and you will be redirected to the Login form.
- 

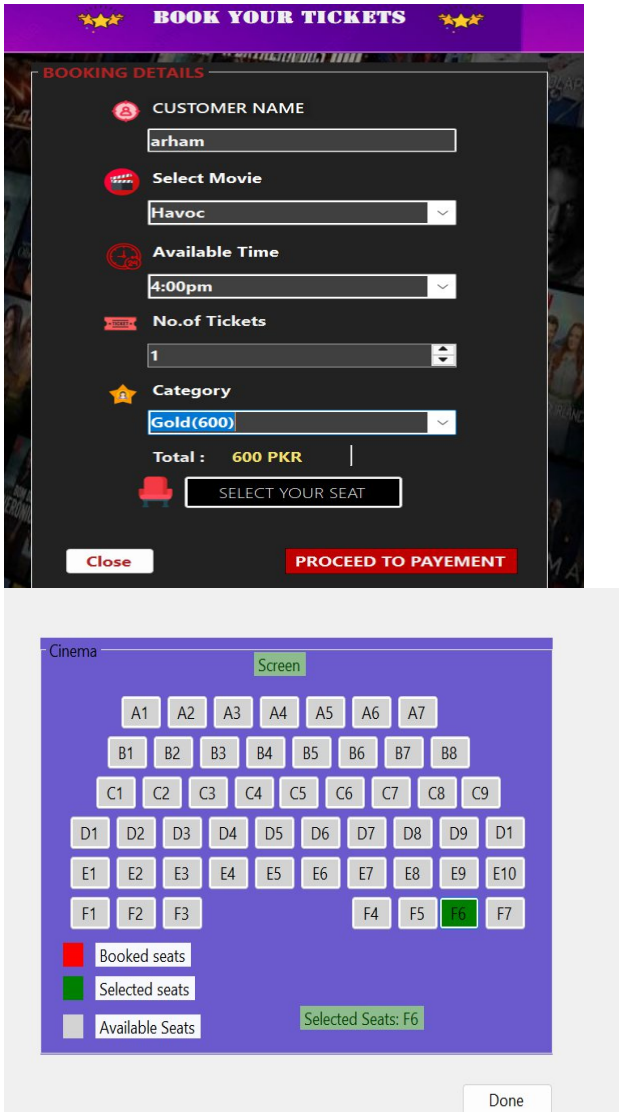
4.3 User Login

- Enter your username and password (case-sensitive) on the Login form.
- Click Login. On success you are taken to the Main Menu (Form3).
- If credentials are wrong, an error message is shown.



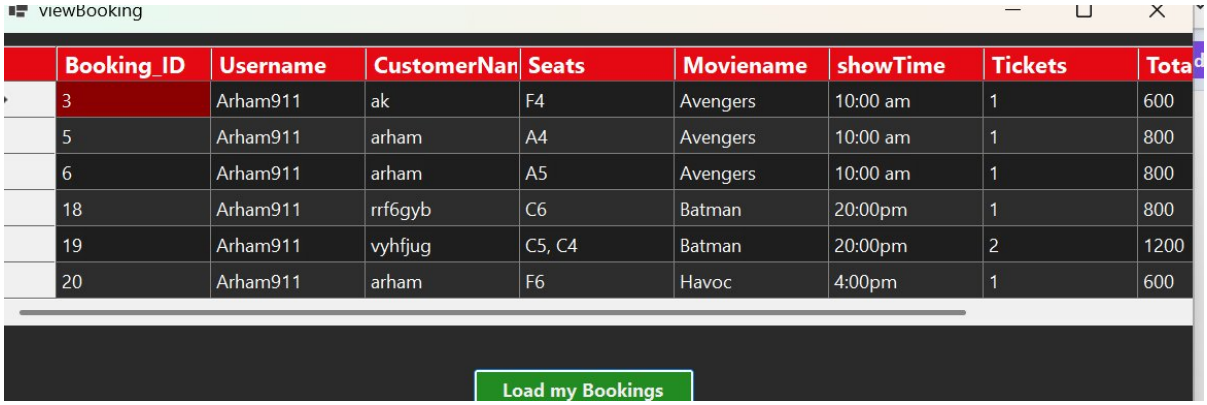
4.4 Booking a Ticket

- From the Main Menu, click New Booking or use the menu item.
- On the Booking Form: enter your name, select a movie, the show time auto-fills.
- Set the number of tickets using the spinner and select a category (VIP/Gold/Silver). The total updates automatically.

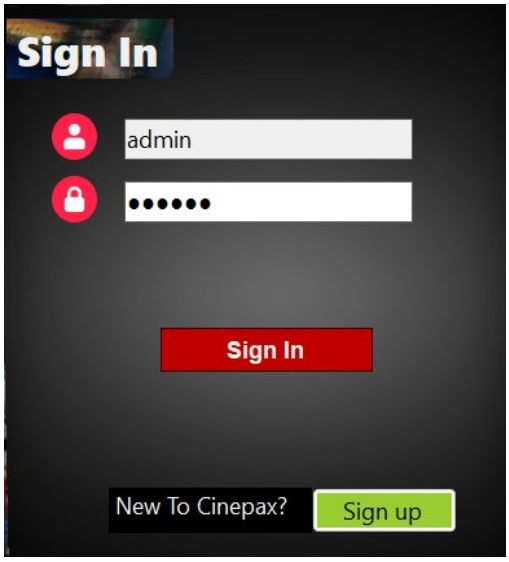
- Click Select Seats. The Seat Selection form opens showing available (gray) and booked (red) seats.
- Click available seats to select them (they turn green). Click again to deselect.
- Once the required number of seats is selected, click Confirm.
- Review details and click Confirm Booking on the Payment form. Select a payment method and click Pay.
- 

4.5 Viewing Bookings

- From the Main Menu, select View Bookings. Your booking history is loaded from the database.



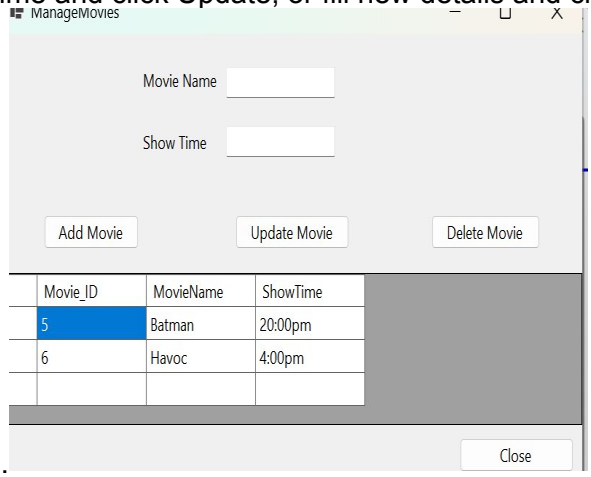
4.6 Admin Login

- On the Login form, enter username admin and password 123123.
- The Admin dashboard opens with menu options: Manage Movies, Manage Bookings, Seat Management.
- 

4.7 Admin – Manage Movies

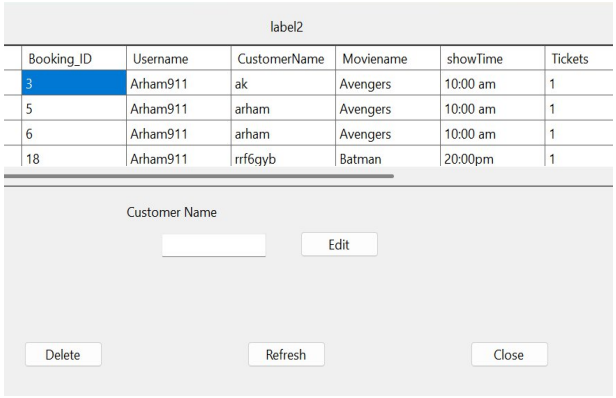
- Click a row in the grid to load the movie details into the text boxes.

- Edit name/time and click Update, or fill new details and click Add, or select a row and click Delete.



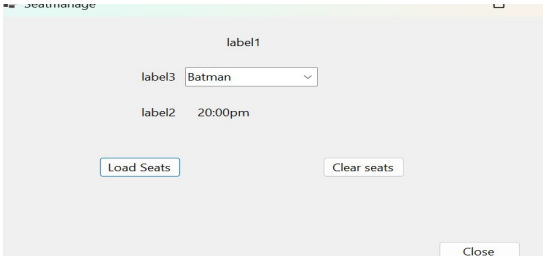
4.8 Admin – Manage Bookings

- All bookings are displayed. Click a row to load the customer name.
- Edit the customer name and click Edit to update, or select a row and click Delete (with confirmation).
-



4.9 Admin – Seat Management

- Select a movie and confirm the time, then click View Seats to see the seat map in read-only mode.
- Click Clear Seats to remove all bookings for the selected movie/time (with confirmation).



REFERENCES

[1] Microsoft Corporation, C# Documentation, https://docs.microsoft.com/en-us/dotnet/csharp/, 2024.

[2] Microsoft Corporation, SQL Server Documentation, https://docs.microsoft.com/en-us/sql/, 2024.

[3] Microsoft Corporation, Windows Forms for .NET, https://docs.microsoft.com/en-us/dotnet/desktop/winforms/, 2024.

[4] I. Sommerville, Software Engineering, 10th ed. Pearson, 2016.

[5] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 8th ed. McGraw-Hill, 2014.